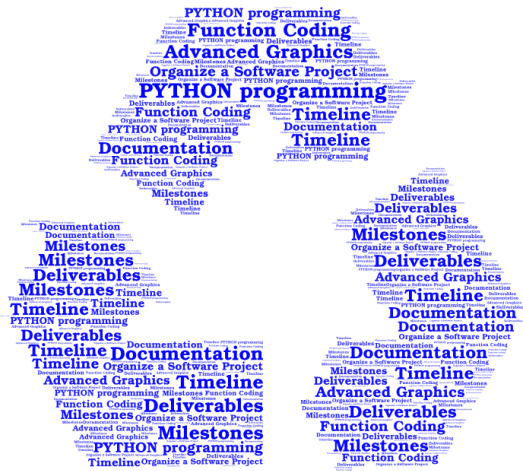




Python Programming: Text encryption

Lecture: Modelling an Simulation

This is a compulsory elective module
of
**Technology of Circular
Economy**
(Master of Engineering)

Dr. Uwe Reimer
Senior Scientist

The Caesar cipher is a shift cipher for text encryption.

What do we need for programming?

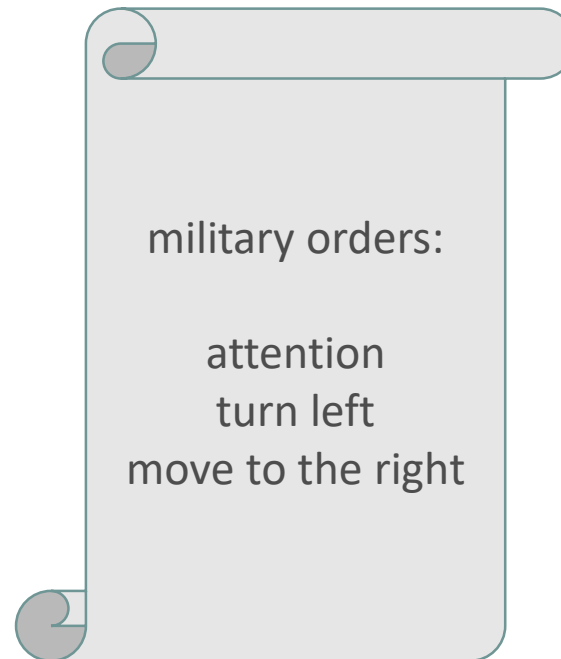
- Strings – built in functions length and slice
- For loop – iterating through strings by position
- Text indent in Python programming
- If statement – an expression that evaluates either true or false
- Files – read text from a file / write text to a file

The Caesar Cipher

The Caesar cipher is named after Julius Caesar (roman emperor, 100 BC – 44 BC), who used it to protect messages of military significance.



picture from <https://www.pngwing.com>



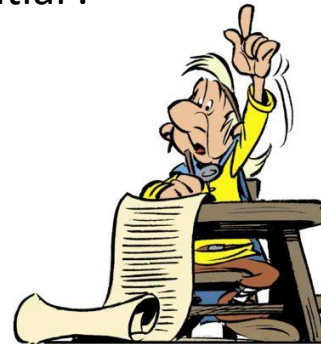
picture from <https://www.pngwing.com>

The Caesar Cipher – a shift cipher

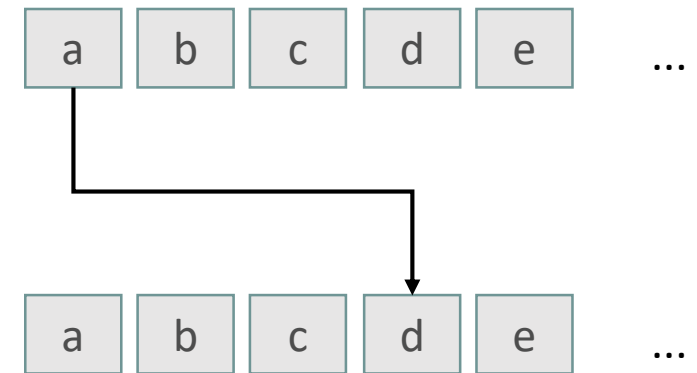
What is a shift cipher?

- Replacement of letters
- Each letter of the alphabet is replaced by a certain shift in position

At the time of the roman empire not every person could read and write. It takes a while to decode the message, if you use only pen and paper. In the past this was a good trade off between 'ease of use' and 'keeping it confidential'.



Example: shift by 3
'a' becomes 'd' in the text



The String data type

The string 'hello'



Positions inside:

[0] [1] [2] [3] [4]

Built in functions:

Length:

b = 'hello'

len(b)

→ returns length of string, here = 5

Slice:

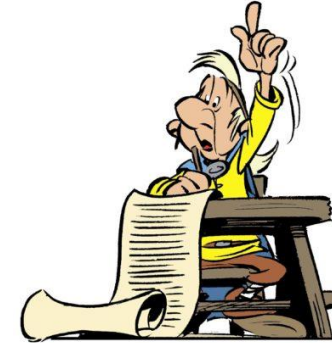
extract text by position

b[0:2] or b[:2]

→ returns text from position 0 up to 2 (but not including 2), here = 'he'

b[2:]

→ returns text from position 2 to end, here = 'llo'



picture from <https://www.pngwing.com>

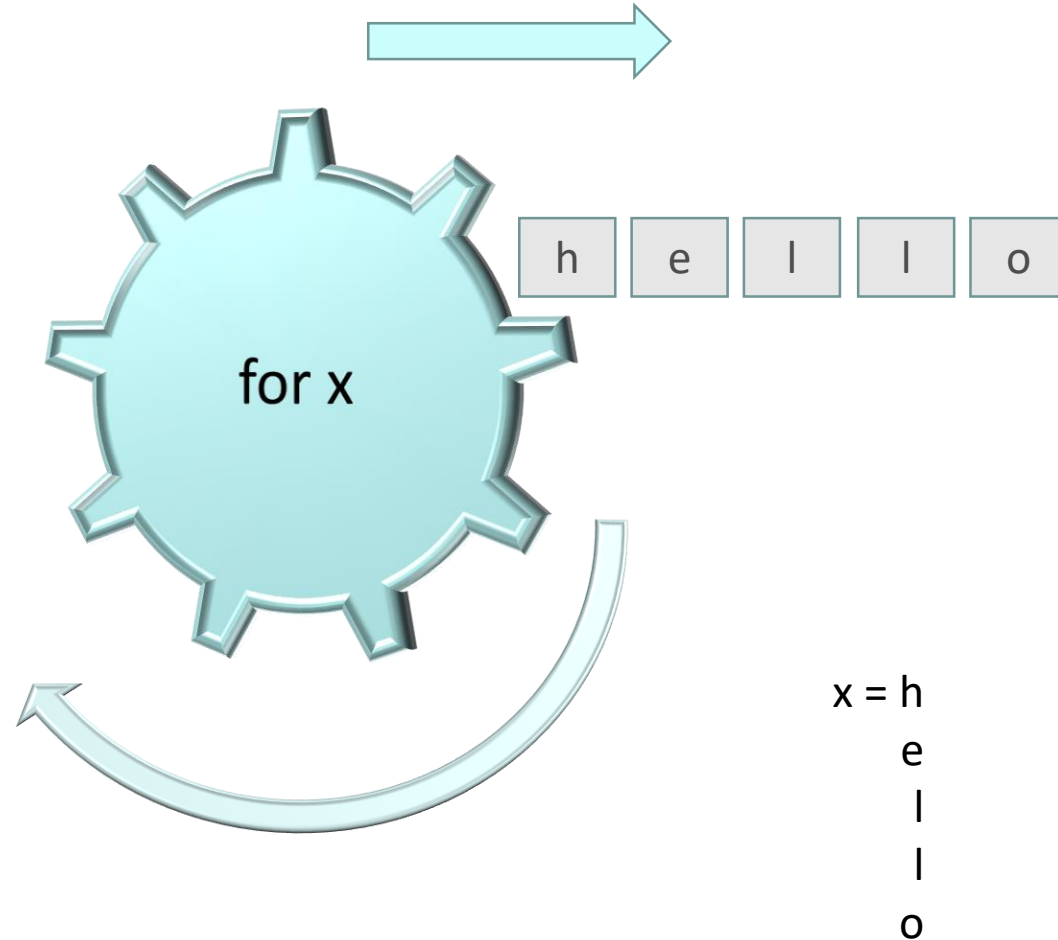
There are many more, but today we need only these two. Check out what else you can find for strings ...

For loop – iterating through a string

```
1 b = 'hello'
2 for x in b:
3     print(x)
4
5
```

Automatically steps through the string.
Here, 'x' holds the value at the respective position.

This works also for other data structures.



While loop – iterating through a string

```
1  b = 'hello'
2
3  c = len(b)
4  x = 0
5
6  while x < c:
7      print( b[x] )
8      x = x + 1
9
```

The 'while' loop is used, if you need to work with the position itself.

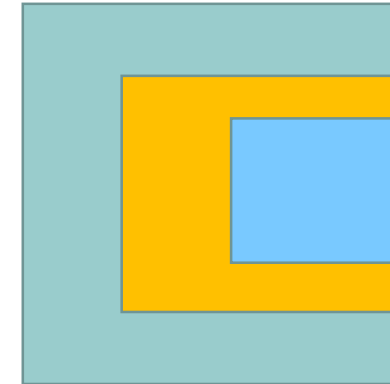
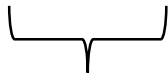
Here, the length is not calculated automatically. You need to provide a condition, at which this loop stops.

Frequent mistake:

Guess what happens if you forget 'x = x + 1' ?

Block indent

```
1  b = 'hello'
2
3  c = len(b)
4  x = 0
5
6  while x < c:
7      print( b[x] )
8      x = x + 1
9
```



Each new program structure (e.g. loops, if conditions, ...) creates an indent. The indent needs to be consistent throughout your program. It can be either a tabulator or several blanks. Normally, your text editor has a default value for this. You may change the default value. If you copy code from other files, then this must remain consistent.

Let's try these two loops.

```
1  b = 'hello'
2  for x in b:
3      print(x)
4
5
```

```
1  b = 'hello'
2
3  c = len(b)
4  x = 0
5
6  while x < c:
7      print( b[x] )
8      x = x + 1
9
```

- 'If' statements are a tool to select actions that should be performed.
- The 'if' statement evaluates a condition.
- There can be several conditions, which are checked one after another.
- The 'else' statement defines a default action, if no other condition holds true.

```
1  b = ['beer', 'wine', 'water', 'juice']
2  c = b[0]
3
4  if c == 'beer':
5      print('YES')
6  else:
7      print('NO')
8
```

Extended examples:

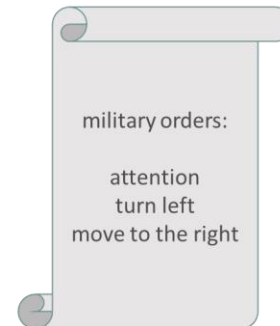
```
1  b = ['beer', 'wine', 'water', 'juice']
2
3  # Say YES to alcohol
4  for c in b:
5      print(c)
6
7      if c == 'beer':
8          print('YES')
9      elif c == 'wine':
10         print('EVEN BETTER')
11     else:
12         print('NO')
```

```
1  b = ['beer', 'wine', 'water', 'juice']
2
3  # Say NO to alcohol
4  for c in b:
5      print(c)
6
7      if c == 'beer':
8          print('NO')
9      elif c == 'wine':
10         print('NO')
11     else:
12         print('YES')
```

What you have learned so far ...

- Strings – built in functions length and slice
- For loop – iterating through strings by position
- Text indent in Python programming
- If statement – an expression that evaluates either true or false
- Files – read text from a file / write text to a file

Now, we are ready for our initial problem:



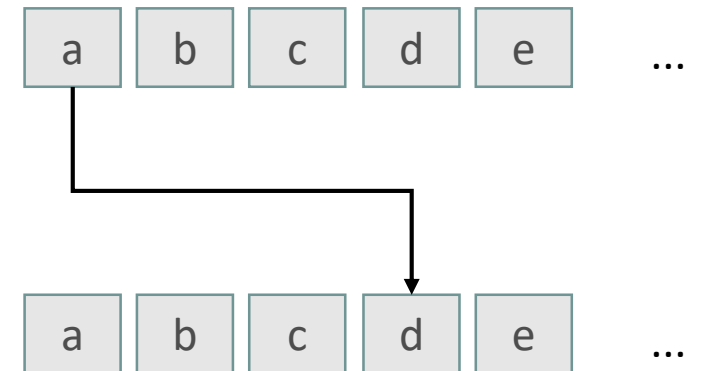
How to code a shift cipher

This is our example text.

'this is an example text for the lecture technology of circular economy!'

How does encoding work?

Example: shift by 3
'a' becomes 'd' in the text



This is our example text.

'this is an example text for the lecture technology of circular economy!'

How does encoding work?

We need to create 'a shifted alphabet'. (An alphabet with shifted positions of characters.)

```
1 mytext = 'this is an example text for the lecture technology of circular economy!'  
2  
3 alorg = 'abcdefghijklmnopqrstuvwxyz'  
4 # shift = 3  
5 alnew = 'defghijklmnopqrstuvwxyzabc'  
6
```

Is that all?

This is our example text.

'this is an example text for the lecture technology of circular economy!'

How does encoding work?

We need to create 'a shifted alphabet'.

We need to add all characters that are not letters. These will not be changed.

```
1 mytext = 'this is an example text for the lecture technology of circular economy!'
2
3 alorg = 'abcdefghijklmnopqrstuvwxyz'
4 # shift = 3
5 alnew = 'defghijklmnopqrstuvwxyzabc'
6 # add all other characters
7 alorg = alorg + ' !'
8 alnew = alnew + ' !'
9
10 print(alorg)
11 print(alnew)
```

How can we code the replacement?

Actually, we create a new text with the modified characters.

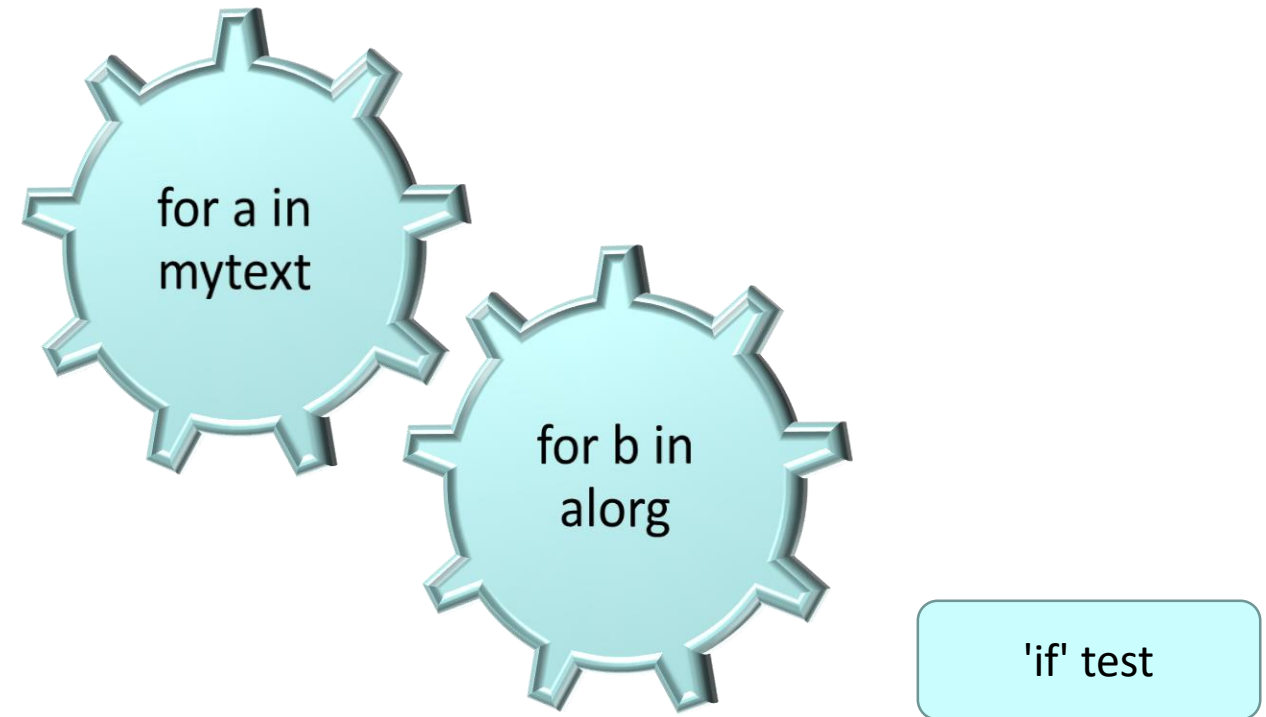
- 1) The starting point is obvious.
Make a loop through our text.



How can we code the replacement?

Actually, we create a new text with the modified characters.

- 1) The starting point is obvious.
Make a loop through our text.
- 2) Search for the position in the original alphabet.
- 3) Use the character at the same position from the shifted alphabet.



```
1 mytext = 'this is an example text for the lecture technology of circular economy!'
2 alorg = 'abcdefghijklmnopqrstuvwxyz'
3 # shift = 3
4 alnew = 'defghijklmnopqrstuvwxyzabc'
5
6 alorg = alorg + ' !'
7 alnew = alnew + ' !'
8
9 newtext = ''
10
11 for a in mytext:
12     # inside this loop, 'a' contains the next letter from the text
13     # now, find the letter in the original alphabet
14     x = 0
15     for b in alorg:
16         if b == a: # letter found at position x
17             newtext = newtext + alnew[x] # new text uses same position in the shift
18             x = x + 1
19
20 print(newtext)
```

The result should read:

'wklv lv dq qhz hadpsoh whaw iru wkh ohfwxuh whfkqrorjb ri flufxodu hfrqrbp!'

Great, congratulations – you can encode messages!

But how can you read them again?

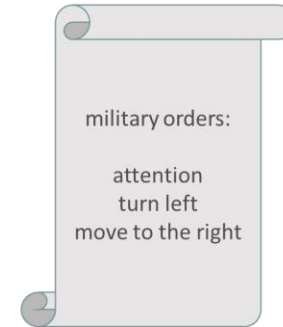
Decoding is actually the same procedure. Now, all we have to do is to do the shift in reverse, i.e. shift by -3.

Let's do it.

Save your program as a new file (new name) and copy the encrypted text into the input string.

```
1 mytext = 'wklv lv dq hadpsoh whaw iru wkh ohfwxuh whfkqrorjb ri flufxodu hfrqrpb!'  
2 alorg = 'abcdefghijklmnopqrstuvwxyz'  
3 # shift = -3  
4 alnew = 'xyzabcdefghijklmnopqrstuvwxyz'  
5  
6 alorg = alorg + ' !'  
7 alnew = alnew + ' !'  
8  
9 newtext = ''  
10  
11 for a in mytext:  
12     # inside this loop, 'a' contains the next letter from the text  
13     # now, find the letter in the original alphabet  
14     x = 0  
15     for b in alorg:  
16         if b == a: # letter found at position x  
17             newtext = newtext + alnew[x] # new text uses same position in the shift  
18             x = x + 1  
19  
20 print(newtext)
```

Now, you can encode and decode text messages ...



... but it would be cool to read the text from a file and write the result into another file.

You can open a file to read content from it or you open an empty file to write something.
The command is:

```
open( 'filename', 'mode' )
```

Examples:

```
myfile1 = open('atext.txt', 'r')      - open the file 'atext.txt' (in the same directory) for reading  
myfile2 = open('btext.txt', 'w')      - create the file 'btext.txt' for writing
```

```
s = myfile1.read()                   - read entire file into string  
s = myfile1.readline()               - read next line (separated by end-line marker)  
a = myfile1.readlines()               - read file into list of strings (for each line)
```

```
myfile2.write(s)                      - write string to file  
myfile2.writelines(a)                 - write all line strings in list to file
```

```
myfile1.close() / myfile2.close()    - close file handler
```

Shift cipher - encoding

```
1  # mytext = 'this is an example text for the lecture technology of circular economy!'  
2  # First, we read text from a file, instead of typing it into the program  
3  
4  myin = open('input-caesar.txt', 'r')  # open a text file in read modus  
5  mytext = myin.read()  # here, we read the entire file into the string  
6  myin.close()  
7  
8  # Finished reading from file. Now start to encode.  
9  
10 alorg = 'abcdefghijklmnopqrstuvwxyz'  
11 # shift = 3  
12 alnew = 'defghijklmnopqrstuvwxyzabc'  
13  
14 alorg = alorg + ' !'  
15 alnew = alnew + ' !'  
16  
17 newtext = ''  
18  
19 for a in mytext:  
20     # inside this loop, 'a' contains the next letter from the text  
21     # now, find the letter in the original alphabet  
22     x = 0  
23     for b in alorg:  
24         if b == a:  # letter found at position x  
25             newtext = newtext + alnew[x]  # new text uses same position in the shifte  
26             x = x + 1  
27
```

```
1  # mytext = 'wklv lv dq hadpsoh whaw iru wkh ohfwxuh whfkqrorjb ri flufxodu hfrqrpb!'  
2  # First, we read text from a file, instead of typing it into the program  
3  
4  myin = open('output-caesar.txt', 'r')  # open a text file in read modus  
5  mytext = myin.read()  # here, we read the entire file into the string  
6  myin.close()  
7  
8  alorg = 'abcdefghijklmnopqrstuvwxyz'  
9  # shift = -3  
10 alnew = 'xyzabcdefghijklmnopqrstuvw'  
11  
12 alorg = alorg + ' !'  
13 alnew = alnew + ' !'  
14  
15 newtext = ''  
16  
17 for a in mytext:  
18     # inside this loop, 'a' contains the next letter from the text  
19     # now, find the letter in the original alphabet  
20     x = 0  
21     for b in alorg:  
22         if b == a:  # letter found at position x  
23             newtext = newtext + alnew[x]  # new text uses same position in the shift  
24             x = x + 1
```


Now, you can encode and decode messages by using the following features in Python:

- Strings – built in functions length and slice
- For loop – iterating through strings by position
- Text indent in Python programming
- If statement – an expression that evaluates either true or false
- Files – read text from a file / write text to a file



Outlook:

It would be cool, if your program can shift the alphabet by any number.

How would you do this?